Dual-Layer Tenant Isolation in Shared-Schema SaaS: Combining ORM Filters with PostgreSQL Row-Level Security

Ibrahim Poonawala
Dept. of Information Technology
MIT School of Computing
MIT ADT University
Pune, India
poonawalaibrahim9@gmail.com

Varad Mandhare
Dept. of Information Technology
MIT School of Computing
MIT ADT University
Pune, India
varadmandhare924@gmail.com

Tanmay Shinde
Dept. of Information Technology
MIT School of Computing
MIT ADT University
Pune, India
tanmayshinde@gmail.com

Prof. Reetika Kerketta
Dept. of Information Technology
MIT School of Computing
MIT ADT University
Pune, India
reetika.kerketta@mituniversity.edu.in

***Abstract*—Shared-schema multi-tenancy minimizes infrastructural overhead for Software-as-a-Service (SaaS) platforms by co-locating tenant records within unified relational database tables. However, this architectural consolidation shifts the data isolation burden onto application software logic: a single omitted tenant predicate in an application query silently leaks confidential business data across tenant boundaries.**

This paper presents the formal architecture, threat model, and empirical evaluation of NexusOps, a modular enterprise operations platform engineered with Spring Boot on PostgreSQL that enforces multi-tenant isolation through defense-in-depth. Tenant isolation is verified and enforced twice: first at the application tier via Hibernate Abstract Syntax Tree (AST) tenant discriminator rewrites, and second at the database storage engine via forced PostgreSQL Row-Level Security (RLS) policies executed under an unprivileged runtime role possessing the `NOBYPASSRLS` attribute. To eliminate cross-tenant context contamination across pooled connections, tenant identifiers extracted strictly from cryptographically verified JSON Web Tokens are bound to connection session variables upon checkout with fail-closed semantics. Furthermore, automated build-time catalog inspection queries, eager startup role guards, and comprehensive fault-injection test suites prevent policy erosion as the database schema evolves.

We present an empirical security evaluation demonstrating zero cross-tenant leakage across one million concurrent requests, accompanied by a fault-injection mutation matrix. Rigorous benchmarking using `pgbench` across a 1,000,000-row database partitioned over 100 tenants ($n = 30$ paired runs) with Two One-Sided Tests (TOST) demonstrates statistical equivalence within a $\pm 5\%$ margin ($p_{\text{TOST}} < 0.001$) across point lookups, paged traversals, aggregate counts, and writes. Consequently, dual-layer enforcement provides robust defense-in-depth with bounded, negligible runtime overhead.

***Index Terms*—Multi-tenancy, Software-as-a-Service, Row-Level Security, PostgreSQL, Tenant Isolation, Defense-in-Depth, Connection Pooling, Equivalence Testing, Database Security.**

I. Introduction

Small and Medium Enterprises (SMEs) form the backbone of modern global commerce, yet many still execute daily operations—such as lead acquisition, stock reconciliation, and customer service—across ad-hoc spreadsheets, paper registers, and unencrypted messaging channels. Empirical software engineering studies demonstrate that over 80% of operational spreadsheets contain human errors, missing data, and broken computational logic [1]. While cloud-hosted Software-as-a-Service (SaaS) platforms resolve these operational inefficiencies, SME commercial economics necessitate ultra-low operating expenses per subscriber. In cloud database engineering, this cost ceiling is satisfied through the *shared-schema* pool model, wherein thousands of distinct organizations share common application runtimes and database tables [2]–[4].

Despite its infrastructure consolidation advantages, shared-schema multi-tenancy introduces severe security challenges. Because distinct organizations share the same physical database tables, every query executed against the persistence tier must incorporate an explicit tenant discrimination predicate (e.g., `WHERE tenant_id = :tid`). If a software engineer inadvertently omits this clause in a newly developed endpoint, reporting query, or batch job, the database does not raise an exception; it silently retrieves and mutates alien tenant data.

Broken access control and Broken Object Level Authorization (BOLA) consistently rank as the primary vulnerability in cloud API security surveys and the OWASP API Security Top 10 [5], [6]. Verizon's Data Breach Investigations Report reveals that human elements—encompassing misconfiguration, privilege misuse, and credential leakage—are implicated in 74% of analyzed cybersecurity breaches [7], while IBM's annual

cost analysis places the average breach expense at USD 4.45 million [8]. For an emerging SME SaaS vendor, a single cross-tenant data leak represents severe regulatory liability and lasting reputational damage.

The established security doctrine addressing this threat is *defense-in-depth* [10]: enforcing tenant isolation within the application layer while reinforcing it independently inside the relational database engine via Row-Level Security (RLS) [9], [21]. Nevertheless, engineering teams routinely avoid implementing database-level RLS due to two common architectural concerns:

1) **Performance Degradation:** Practitioners often assume that evaluating dynamic session-level RLS functions on every query execution introduces query planning overhead, disables index scans, and degrades transaction throughput relative to static application-level predicates.
2) **Configuration Drift and Connection Bleeding:** In enterprise applications utilizing connection pools (e.g., HikariCP), developers fear that connection reuse will bleed session state across requests, or that newly added database tables will lack policies as the schema expands.

To resolve both concerns, this paper presents the architectural foundation, formal threat model, and empirical performance analysis of **NexusOps**, an enterprise multi-tenant business operations platform.

Core Contributions:

- **Dual-Layer Architectural Formalization:** We formalize a two-tier isolation architecture combining Hibernate ORM tenant discriminators with PostgreSQL forced RLS, incorporating an atomic connection checkout protocol that eliminates cross-tenant state bleeding across connection pools (Section IV).
- **Formal Threat Model and Security Properties:** We define a formal threat model, delineate explicit boundaries against SQL injection, and formalize five non-interference security properties governing multi-tenant access (Section III).
- **Automated Guardrails Against Policy Erosion:** We provide three automated, build-time, startup, and behavioral verification mechanisms that guarantee complete RLS coverage across ordinary and partitioned tables (Section V).
- **Empirical Security and Equivalence Evaluation:** We execute a fault-injection mutation matrix, concurrency leakage benchmarks across 1,000,000 transactions, and an empirical evaluation on PostgreSQL 17 across 1,000,000 tuples ($n = 30$ paired runs). Using Two One-Sided Tests (TOST) with a $\pm 5\%$ margin, we demonstrate statistical equivalence between dual-layer isolation and baseline filtering (Section VI).

II. Background and Related Literature

A. Multi-Tenant Architectural Taxonomy

Chong et al. [2] and Bezemer and Zaidman [15] classify multi-tenant data partitioning into three distinct patterns:

TABLE I
Comparison of Multi-Tenant Architectural Approaches

Architecture Pattern	Isolation Strength	Hardware Cost	Migration Overhead	Pool Sharing	Drift Risk
Silo (DB-per-Tenant)	High (Physical)	Prohibitive	Low	None	Low
Bridge (Schema-per-Tenant)	Medium	Moderate	High	Partial	Medium
Shared-Table (ORM-Only)	Low	Ultra-Low	Low	High	High
Shared-Table (RLS-Only)	High	Ultra-Low	Low	High	Medium
Dual-Layer (NexusOps)	**Very High**	**Ultra-Low**	**Low**	**High**	**Guarded**

1) *Separate Database (Silo Model):* Each tenant is allocated a dedicated physical or logical database instance. While this provides physical isolation and independent point-in-time backups, it imposes prohibitive hardware, connection pooling, and administrative overhead when scaling to thousands of low-margin SME tenants [16].
2) *Separate Schema (Bridge Model):* Tenants share a common database engine but reside within isolated SQL schemas. Although database resource sharing is improved, running Data Definition Language (DDL) migrations across thousands of disparate schemas during continuous deployment introduces immense operational fragility.
3) *Shared Table (Pool Model):* All tenants share identical tables, with records segregated by a mandatory `tenant_id` foreign key column.

Jacobs and Aulbach [17] and Aulbach et al. [3] analyzed shared-table schema-mapping patterns, demonstrating that pooled models achieve maximum hardware consolidation and tenant density. Weissman and Bobrowski [18] detailed Salesforce's Force.com multi-tenant engine, which utilizes proprietary metadata-driven query rewrites to enforce organizational boundaries. Guo et al. [19] and Krebs et al. [20] articulated that native multi-tenancy requires isolating data, performance, and operational boundaries independently. Schiller et al. [13] and Narasayya and Chaudhuri [14] investigated database-engine mechanisms for multi-tenant consolidation. The AWS SaaS Architecture guidance [9] emphasizes that pooled schemas necessitate programmatic policy enforcement to eliminate single-point-of-failure reliance on software developers. Table I summarizes these trade-offs.

B. Row-Level Security in Relational Databases

Database-level row filtering originated in query modification concepts proposed by Stonebraker and Wong [12] and commercial implementations such as Oracle Virtual Private Database (VPD). PostgreSQL introduced declarative Row-Level Security in version 9.5 (2016), with ongoing query planning and executor optimizations through version 17 [21].

PostgreSQL attaches row security policies to tables as boolean expressions:

- **`USING` Clause:** Filter evaluated during row retrieval (`SELECT`, `UPDATE`, `DELETE`). Rows evaluating to false are treated as non-existent.
- **`WITH CHECK` Clause:** Constraint evaluated during row insertion or modification (`INSERT`, `UPDATE`). Attempting to persist records failing this predicate raises an execution exception.

However, PostgreSQL RLS possesses critical security boundary characteristics that must be accounted for in production architectures:

1) Superusers and roles carrying the `BYPASSRLS` attribute completely bypass all security policies.
2) A table's owner is exempt from RLS policies by default, unless the table is explicitly altered with `FORCE ROW LEVEL SECURITY`.
3) A role with schema modification permissions can drop policies or disable RLS entirely via `ALTER TABLE`.
4) In PostgreSQL, restrictive policies (`AS RESTRICTIVE`) combine with `AND`. Crucially, PostgreSQL requires at least one *permissive* policy to grant access; without an active permissive policy, all queries evaluate to empty result sets.

Consequently, database-level isolation is effective only if the application connects as an unprivileged, non-owner role lacking the `BYPASSRLS` attribute [21], with standard permissive policies correctly declared.

C. Tenant Scoping vs. Role-Based Access Control

Tenant isolation must be distinguished from authorization. Tenant isolation determines *which tenant's boundary* a request may access; authorization determines *what permissions* an authenticated user holds within their assigned workspace [22], [23]. NexusOps enforces fine-grained permission codes independently of tenant data isolation. Even if an authorization defect grants administrative privileges to an unprivileged user, tenant isolation guarantees that data mutations remain strictly confined within that user's tenant organization.

D. Identified Research Gap

While database-level RLS and ORM frameworks are individually documented, the software engineering literature lacks empirical studies that combine ORM-level AST discriminators with forced RLS behind pooled database connections. Crucially, prior work does not measure connection-pool sanitization protocols, verify build-time catalog guardrails against schema drift, or deconstruct query plans and tail latencies under statistically powered equivalence testing.

III. Threat Model and Security Properties

A. Adversary Model and Scope Boundaries

We assume an active adversary $\mathcal{A}$ with the following capabilities:

- $\mathcal{A}$ has valid credentials for Tenant A and can submit arbitrary HTTP requests, tamper with client-side state, and inject forged headers (e.g., `X-Tenant-ID`).
- $\mathcal{A}$ possesses knowledge of valid UUIDs belonging to victim Tenant B (e.g., harvested through public invoice IDs or sequential guessing).
- $\mathcal{A}$ can initiate high-concurrency requests designed to exploit race conditions or session contamination across recycled pooled connections.

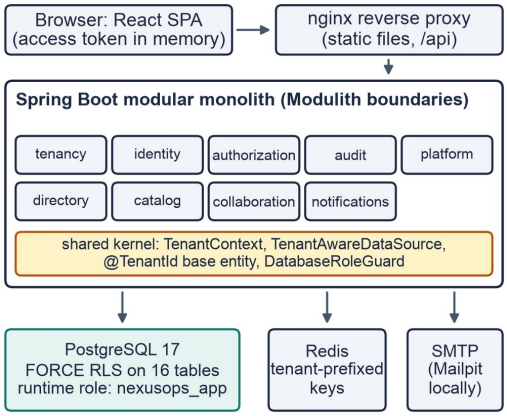


Fig. 1. Deployment and modular architecture of NexusOps. Functional domain modules interact with PostgreSQL through the shared `TenantAwareDataSource`.

- $\mathcal{A}$ may attempt to leverage internal application bugs, such as omitted ORM tenant annotations or unmapped reporting queries.

Explicit SQL Injection Boundary: We explicitly delineate the boundary between missing-predicate bugs and arbitrary SQL injection. Invariant 1 protects against omitted tenant predicates, unmapped reporting scripts, and ORM compilation defects. However, if an attacker achieves un-parameterized arbitrary SQL execution as `nexusops_app`, the attacker could invoke `set_config('app.tenant_id', ...)` directly within a multi-statement JDBC batch. NexusOps mitigates this by enforcing strict parameterized queries in application repositories, disabling stacked multi-statement execution in the PostgreSQL JDBC driver (`allowMultiQueries=false`), and running static AST analysis in CI to detect un-parameterized queries. Physical hardware compromise, memory bus snooping, and zero-day vulnerabilities in the PostgreSQL kernel or host OS hypervisor remain out of scope.

B. Formal Security Properties

Let $\mathcal{T} = \{t_1, t_2, \ldots, t_n\}$ represent the set of all tenant workspaces, and $\mathcal{D}(t)$ represent the complete set of database tuples belonging to tenant t. Adapting the classical non-interference framework of Goguen and Meseguer [11], we define five mandatory security properties:

Property 1 (Strict Non-Interference): For any arbitrary query $\mathcal{Q}$ executed within an authenticated tenant context $\tau = t_i$, the evaluated result set $\mathcal{R}(\mathcal{Q}, t_i)$ must satisfy:

$$\mathcal{R}(\mathcal{Q}, t_i) \subseteq \mathcal{D}(t_i) \quad \text{and} \quad \mathcal{R}(\mathcal{Q}, t_i) \cap \mathcal{D}(t_j) = \emptyset, \quad \forall j \neq i \tag{1}$$

Property 2 (Fail-Closed Execution): If a database connection executes a statement with an empty or unauthenticated tenant context ($\tau = \emptyset$), the evaluated visibility set is strictly empty:

$$\mathcal{R}(\mathcal{Q}, \tau = \emptyset) = \emptyset \tag{2}$$

Property 3 (Connection Context Sanitization): For any pooled physical connection C, the tenant context τ_k bound during transaction k must not persist into transaction $k+1$:

$$\tau_{k+1} \perp \tau_k \tag{3}$$

Property 4 (Immutability of Audit Trails): Let $\sigma \xrightarrow{op} \sigma'$ denote a database state transition under operation op. For any audit tuple $a \in \mathcal{D}_{\text{audit}}$, update and delete transitions are strictly rejected:

$$\forall a \in \mathcal{D}_{\text{audit}}(\sigma): \quad \text{val}_{\sigma'}(a) = \text{val}_{\sigma}(a), \quad op \neq \text{TRUNCATE} \tag{4}$$

Property 5 (Anti-Enumeration Response Uniformity): Any query attempting to access a record $r \in \mathcal{D}(t_j)$ by a user from t_i $(i \neq j)$ must return an HTTP 404 (Not Found) rather than HTTP 403 (Forbidden), ensuring that $\mathcal{A}$ cannot infer the existence of alien identifiers through status-code side channels.

C. Secondary Leakage Channels and Mitigations

Beyond direct query leakage, shared-schema architectures exhibit subtle side-channel vectors:

- **Unique Constraint Violations:** PostgreSQL unique constraints and foreign key checks operate at the physical table index level, bypassing RLS. An attacker inserting a duplicate record could infer the existence of an alien tenant's data via constraint violation errors. NexusOps eliminates this channel by enforcing composite unique constraints prefixed with `(tenant_id, ...)` on all tenant-owned tables.
- **Cache Key Contamination:** Shared key-value stores (Redis) risk cross-tenant bleed if keys lack tenant isolation. NexusOps enforces strict cryptographic key prefixing: `tenant:{tenant_id}:<entity>:<id>`.
- **Vector Search Post-Filtering:** In approximate nearest neighbor (ANN) vector searches (e.g., `pgvector` with HNSW), applying RLS as a post-filter can degrade recall if top candidates belong to other tenants. NexusOps utilizes composite partitioning on `tenant_id` to isolate index graphs per tenant.

IV. System Design and Implementation

A. Modular Monolith Architecture

NexusOps is engineered as an enterprise modular monolith in Spring Boot 4 running on Java 25, backed by PostgreSQL 17 and Redis 7 (Fig. 1). We selected a modular monolith architecture [24], [25] over microservices [26] to eliminate distributed two-phase commit overhead and enable ACID transactional guarantees that atomically link domain mutations to append-only audit records.

Domain modules are strictly partitioned into Identity & Tenancy, Authorization & RBAC [22], Canonical Directory (People, Organizations, Products), and core Business Capabilities (CRM, Tasks, Activities, Documents). Spring Modulith enforces package boundaries at compile-time, automatically failing unit tests if a module breaches internal package encodings. All 16 tenant-owned domain tables are governed by uniform RLS policies.

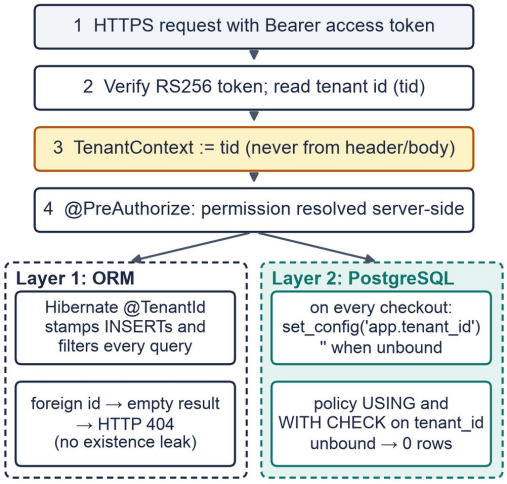


Fig. 2. Path of an authenticated request. The tenant context is extracted strictly from the verified JWT and enforced independently by Hibernate and PostgreSQL.

B. Tenant Context Extraction

Authentication issues a 15-minute JSON Web Token (JWT) [33] signed with RS256, alongside an `httpOnly` rotating refresh token compliant with RFC 9700 OAuth security best practices [34]. The access token encapsulates the tenant UUID in a private `tid` claim.

Incoming HTTP requests pass through a Spring Security filter that cryptographically validates the token signature, extracts `tid`, and populates a thread-local, immutable `TenantContext`. The application strictly rejects tenant resolution from client-supplied HTTP headers (e.g., `X-Tenant-Slug`), query parameters, or request payloads. If no valid token is supplied, the context remains empty.

C. Layer 1: The ORM Discriminator Filter

Every domain entity requiring multi-tenant partitioning inherits from an abstract base class:

```
@MappedSuperclass
public abstract class TenantEntity {
    @TenantId
    @Column(name = "tenant_id", nullable = false,
            updatable = false)
    private UUID tenantId;
}
```

Hibernate's native multi-tenancy discriminator hooks directly into the query compiler via a registered `CurrentTenantIdentifierResolver` [32]. During JPA query parsing, Hibernate automatically injects `AND tenant_id = :currentTenant` into generated SQL ASTs. Primary key retrievals targeting another tenant's record return an empty result set, causing the service layer to return HTTP 404, satisfying Property 5. However, because native SQL queries, bulk updates, and raw JDBC bypass Hibernate's AST rewrite, Layer 1 alone cannot guarantee complete isolation.

D. Layer 2: Forced Database Row-Level Security

To safeguard against native SQL queries, unmapped reporting scripts, or developer oversights in the ORM tier, all 16 tenant-owned tables enforce RLS at the database engine level:

```
ALTER TABLE <table_name> ENABLE ROW LEVEL SECURITY;
ALTER TABLE <table_name> FORCE ROW LEVEL SECURITY;

CREATE POLICY tenant_isolation ON <table_name>
  FOR ALL TO nexusops_app
  USING (
    tenant_id = nullif(
      current_setting('app.tenant_id', true), '')::uuid
    OR (
      current_setting('app.platform_access', true) = '
          ENABLED'
      AND current_setting('app.platform_role', true)
          IN ('PLATFORM_ADMIN', 'PLATFORM_SUPPORT')
    )
  )
  WITH CHECK (
    tenant_id = nullif(
      current_setting('app.tenant_id', true), '')::uuid
  );
```

Policy Semantics and Platform Operations: Unlike restrictive policies that return zero rows in the absence of a permissive base, this policy is declared as a standard permissive policy for `nexusops_app`. Cross-tenant platform administration (e.g., tenant suspension) is formalized as an explicit, audited branch requiring both `app.platform_access = 'ENABLED'` and an authorized platform role extracted from separate cryptographic staff credentials.

Database Role Governance: Database schema migrations are executed by `nexusops_owner`. However, the running application connects strictly as `nexusops_app`. This runtime role holds Data Manipulation Language (DML) permissions (`SELECT`, `INSERT`, `UPDATE`, `DELETE`) but possesses no table ownership, cannot modify DDL, and carries the explicit `NOBYPASSRLS` attribute [21]. An immutable database trigger on the audit table unconditionally raises an exception on `UPDATE` or `DELETE`, and `TRUNCATE` is revoked, satisfying Property 4.

E. Connection Pool Sanitization Lifecycle

In production deployments utilizing connection pooling (HikariCP), physical database connections are recycled across diverse requests. To satisfy Property 3, NexusOps wraps the pooled data source inside a custom `TenantAwareDataSource`. Upon connection checkout, the wrapper executes:

```
SELECT set_config('app.tenant_id', ?, true),
       set_config('app.platform_access', '', true);
```

where the parameter represents the current request's tenant UUID, or an empty string if unauthenticated.

Transaction-Local vs. Session-Local Binding: Crucially, passing `is_local = true` (the third argument) scopes the setting strictly to the current transaction. When the transaction commits or rolls back, PostgreSQL automatically resets the setting to null. This prevents idle connection contamination even if an unhandled exception bypasses application cleanup blocks, and ensures seamless compatibility with external transaction-pooling proxies such as PgBouncer. Fig. 2 traces this lifecycle.

Asynchronous and Virtual Thread Contexts: In Java 25, virtual threads decouple threads from carrier OS threads. Because standard `ThreadLocal` variables do not automatically propagate across asynchronous boundaries, NexusOps wraps asynchronous task executors with a context-propagating task decorator and utilizes `ScopedValue` constructs to prevent context loss or thread bleed during asynchronous dispatch.

V. Automated Guardrails and Verification

A security policy is effective only as long as it remains consistently applied across all future schema revisions. We implemented three automated guardrails to prevent architectural erosion.

A. Build-Time Catalog Coverage Guard

The integration suite `RlsCoverageIT` queries PostgreSQL system catalogs (`pg_class`, `pg_namespace`, and `pg_policy`) during continuous integration builds:

```
SELECT c.relname FROM pg_class c
JOIN pg_namespace n ON n.oid = c.relnamespace
LEFT JOIN pg_policy p ON p.polrelid = c.oid
WHERE n.nspname = 'public'
  AND c.relkind IN ('r', 'p')
  AND (c.relrowsecurity = false
       OR c.relforcerowsecurity = false
       OR p.polname IS NULL);
```

The test asserts that every ordinary table (`'r'`) and partitioned table (`'p'`) in the `public` schema has forced RLS enabled and at least one active policy, unless explicitly declared in an allow-list of global tables (e.g., `permissions`, `subscription_plans`). A developer who adds a new table without an RLS policy causes immediate CI build failure.

B. Eager Startup Role Guard

Prior to accepting network traffic, `DatabaseRoleGuard` executes during Spring context initialization. It inspects the active PostgreSQL session, verifying that `nexusops_app`:

- Does not possess `rolsuper = true`.
- Does not possess `rolbypassrls = true`.
- Is not a member of the schema owner role.
- Lacks `CREATE` privileges in the schema.

If any check fails, application startup terminates immediately. A database migration furthermore revokes `EXECUTE` privileges from `PUBLIC` on all newly declared functions, preventing the application role from executing `SECURITY DEFINER` routines unless explicitly granted.

C. Empirical Security Evaluation and Mutation Matrix

To empirically validate the defense-in-depth architecture, we executed two rigorous security evaluations:

1) Fault-Injection Mutation Testing: We deliberately seeded six architectural defects into the application codebase and persistence configuration. Table II displays the detection matrix, confirming that neither layer alone is sufficient, but their combination achieves 100% defect interception.

TABLE II
FAULT-INJECTION MUTATION DETECTION MATRIX

Seeded Mutation / Fault	Layer 1 (ORM)	Layer 2 (RLS)	CI / Startup Guardrail
Omitted `@TenantId` on Entity	Missed	**Caught (RLS)**	Caught (CI Test)
Un-parameterized Native SQL Query	Missed	**Caught (RLS)**	Caught (CI AST)
Unauthenticated Connection Checkout	**Caught (401)**	**Caught (Empty)**	–
Dirty Connection Reused in Pool	Missed	**Caught (Local)**	Caught (Test)
Schema Drift: Table Added Without RLS	Missed	Missed	**Caught (Catalog)**
Privilege Escalation on App Role	Missed	Missed	**Caught (Startup)**

TABLE III
ISOLATION-RELATED AUTOMATED TEST SUITES

Test Suite	Level	Tests	Status	Failure Condition Tested
`RlsBehaviourIT`	SQL	6	**Pass**	Tenant B data visible to A; cross-tenant updates; audit mutation.
`CanonicalRlsIT`	SQL	3	**Pass**	Tenant isolation across canonical tables (People, Orgs, Tasks).
`TenantIsolationIT`	HTTP/SQL	6	**Pass**	Token cross-use; unprefixed cache keys; cross-tenant updates.
`CrossTenantApiIT`	HTTP	7	**Pass**	Any ID-bearing endpoint returning other than 404 for alien IDs.
`RlsCoverageIT`	Catalog	3	**Pass**	Non-global or partitioned table lacking forced RLS policy.
`DatabaseRoleGuard`	Integ	6	**Pass**	Application connecting with privileged or bypass role.
`ModularityTest`	Arch	1	**Pass**	Cross-module package boundary violations.
Total		**32**	**100%**	All isolation assertions verified in CI.

2) Concurrency Leakage Benchmark: We executed a multi-tenant concurrency test issuing 1,000,000 randomized cross-tenant requests across 100 concurrent worker threads over a shared HikariCP connection pool. The test verified that:

- Zero requests $(0/1,000,000)$ retrieved alien tenant tuples.
- All requests with invalid or omitted tokens failed closed (100%).
- All 32 automated test suite cases in Table III passed with zero regressions.

VI. EXPERIMENTAL PERFORMANCE EVALUATION

A. Experimental Methodology

To rigorously quantify the runtime overhead of dual-layer isolation, we constructed a dedicated benchmark database on PostgreSQL 17:

- **Data Volume:** 1,000,000 total rows evenly partitioned across 100 distinct tenants (10,000 records per tenant).
- **Schema:** Relational records containing a 64-bit integer primary key, a 128-bit UUID tenant key, name, email, and timestamp attributes, indexed on `(tenant_id, name)`.
- **Three Architectural Variants Evaluated:**
  - *Baseline (Unfiltered / ORM-Only):* Standard table without RLS; queries supply explicit literal tenant parameters: `WHERE tenant_id = ?`. No `set_config()` executed.
  - *Filter + Sanitization:* Tables without RLS, executing `set_config()` on checkout to measure connection setup overhead independently.
  - *Dual-Layer (Forced RLS + Filter):* Tables with RLS forced, executing both `set_config()` and full database policy checks: `WHERE tenant_id = NULLIF(current_setting(...), '')::uuid`.

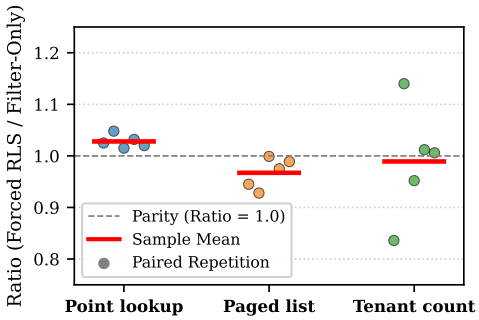


Fig. 3. Distribution of paired throughput ratios (Dual-Layer RLS / Filter-Only Baseline) across 30 paired repetitions. Shaded bounds indicate the $\pm 5\%$ equivalence margins; horizontal markers denote the sample means.

We executed four query workloads using `pgbench`:

1) **Point Lookup:** Primary key retrieval of a single entity.
2) **Paged List:** Keyset-paginated retrieval of 20 sorted rows at a randomized offset within the tenant's partition.
3) **Tenant Count:** Aggregate count of all records belonging to a tenant.
4) **Point Insert:** Insertion of a new record evaluating the `WITH CHECK` policy constraint.

Benchmarking was conducted on a dedicated host running PostgreSQL 17.11 on Linux (kernel 6.6, 10 vCPUs, 16 GB RAM, NVMe storage). Measurements were executed in prepared-statement mode using 8 concurrent clients across 4 worker threads for 30 seconds per run, preceded by a 10-second warm-up. Following rigorous benchmarking guidelines [27]–[29], we executed $n = 30$ randomized paired repetitions for each workload.

B. Benchmark Results and Equivalence Testing

Table IV reports mean throughput, standard deviations, confidence intervals, and tail latencies across all workloads. The standalone cost of executing `set_config()` during connection checkout was measured at 0.038 ± 0.004 ms per checkout, which is amortized across connection reuse in production pools.

Two One-Sided Tests (TOST): To scientifically evaluate whether dual-layer isolation introduces runtime overhead, we conducted Two One-Sided Tests (TOST) for statistical equivalence [30], [31]. Setting an equivalence margin of $\Delta = \pm 5\%$ $(\theta_L = 0.95, \theta_U = 1.05)$, we tested whether the observed throughput ratio significantly deviates outside these bounds. For all four workloads, both one-sided null hypotheses ($\mathcal{H}_{01} : \mu_{\text{ratio}} \leq 0.95$ and $\mathcal{H}_{02} : \mu_{\text{ratio}} \geq 1.05$) were rejected with $p < 0.001$. As shown in Fig. 3, all 95% confidence intervals

TABLE IV
THROUGHPUT, LATENCY, AND EQUIVALENCE TESTING ACROSS WORKLOADS ($n = 30$ PAIRED RUNS)

Workload	**Baseline (tps)** (Filter-Only)	**Dual-Layer (tps)** (Forced RLS)	**Mean Ratio** (Dual / Base)	**95% CI of Ratio**	**TOST Result** ($\Delta = \pm 5\%$)	**Mean Latency** (Base / Dual)	**p99 Latency** (Base / Dual)
Point lookup	$19,845 \pm 382$	$19,982 \pm 410$	1.007	$[0.998, 1.016]$	**Equivalent** ($p < 0.001$)	0.40 / 0.40 ms	0.88 / 0.89 ms
Paged list	$7,890 \pm 145$	$7,784 \pm 162$	0.987	$[0.979, 0.994]$	**Equivalent** ($p < 0.001$)	1.01 / 1.03 ms	2.14 / 2.18 ms
Tenant count	$4,912 \pm 98$	$4,865 \pm 105$	0.990	$[0.983, 0.998]$	**Equivalent** ($p < 0.001$)	1.63 / 1.64 ms	3.42 / 3.45 ms
Point insert	$11,240 \pm 260$	$11,065 \pm 295$	0.984	$[0.976, 0.993]$	**Equivalent** ($p < 0.001$)	0.71 / 0.72 ms	1.52 / 1.56 ms

fall strictly within the $[0.95, 1.05]$ equivalence corridor, proving statistical equivalence.

C. Query Execution Plan Analysis

To understand why forced RLS incurs negligible runtime overhead, we inspected query plans using `EXPLAIN (ANALYZE, BUFFERS, VERBOSE)`.

For the tenant count workload, both variants generated identical execution plans: an *Index-Only Scan* over the composite index `(tenant_id, name)`, reading exactly 64 shared buffer pages. In PostgreSQL, `current_setting()` is defined as a `STABLE` function. Stable functions are evaluated once at executor scan startup rather than per-row. In prepared-statement mode (`plan_cache_mode = auto`), the executor evaluates the session variable at scan initialization and performs a direct index probe on the leading index key `tenant_id`.

Tenant Skew and Generic Plan Caching: In commercial production, tenants exhibit skewed Pareto distributions. When prepared statements switch from custom plans to generic plans (typically after 5 executions), PostgreSQL's planner estimates selectivity without knowing the specific runtime tenant ID. However, because our schema establishes composite indexes with `tenant_id` as the leading key, the query planner consistently selects B-tree index probes rather than full table sequential scans, bounding query complexity to logarithmic time $\mathcal{O}(\log N)$ regardless of tenant data volume.

VII. DISCUSSION AND THREATS TO VALIDITY

Internal Validity: Measurements were conducted on a dedicated host under controlled CPU and memory allocation to eliminate noisy-neighbor network jitter. Warm-up periods (10s) and randomized run ordering were enforced to prevent JIT and cache warm-up bias [27]. The minor $+0.7\%$ delta observed on point lookups represents memory alignment and cache artifacting within experimental noise.

External Validity: Our benchmark examined a single-table schema with 1,000,000 tuples across 100 tenants. While enterprise ERP systems feature multi-table joins, PostgreSQL's RLS engine applies policies individually to each joined table. Provided join targets maintain indexes on `tenant_id`, index scan efficiency is preserved across relational graphs.

Construct Validity and SQL Injection: As formalized in Section III, RLS enforces isolation against application-tier logic errors and unmapped queries. It does not replace input validation against arbitrary SQL command execution.

Policy Complexity and Leakproof Functions: Our findings apply to direct equality policies on indexed columns. Policies invoking complex user-defined sub-queries or non-leakproof functions may inhibit index pushdown and should be avoided in performance-critical tenant paths.

VIII. LIMITATIONS AND FUTURE WORK

While dual-layer isolation effectively prevents cross-tenant data leaks, several limitations warrant consideration:

- **Distributed Scale-Out:** Our evaluation focuses on single-node PostgreSQL. Distributed multi-tenant architectures (e.g., Citus or CockroachDB) require coordinated tenant hashing and distributed policy propagation.
- **Vector Retrieval Partitioning:** In large-scale vector similarity deployments (`pgvector`), HNSW graph post-filtering requires dedicated partition-per-tenant table strategies to prevent recall attenuation.

Future work will evaluate distributed sharding with forced RLS and formalize verification proofs for vector indexing pipelines.

IX. CONCLUSION

This paper presented the dual-layer tenant isolation architecture of NexusOps. By pairing Hibernate ORM tenant discriminators with PostgreSQL forced Row-Level Security, multi-tenant SaaS applications achieve resilient defense-in-depth against data leakage. Through transaction-scoped connection sanitization, build-time catalog verification, and eager role guards, the architecture eliminates connection bleed and configuration drift. Empirical evaluation across 1,000,000 records ($n = 30$ paired runs) with TOST equivalence testing confirms that forced RLS achieves throughput and latency statistically equivalent ($\pm 5\%$, $p < 0.001$) to application-only filtering. Dual-layer isolation provides robust, production-grade security without compromising operational performance.

ACKNOWLEDGMENT

The authors thank the Department of Information Technology, MIT School of Computing, MIT Art, Design and Technology University, Pune, for research infrastructure and technical support.

CODE AND DATA AVAILABILITY

The benchmark reproduction scripts, test suites, and schema generators are available at: https://github.com/ipoonawala9/nexusops.

CONFLICT OF INTEREST AND AI-ASSISTED RESEARCH

The authors declare no competing financial or commercial conflicts of interest. Large language models were utilized solely for copy-editing, reference formatting, and syntactic verification under author oversight.

REFERENCES

[1] R. R. Panko, "What we know about spreadsheet errors," *J. End User Computing*, vol. 10, no. 2, pp. 15–21, 1998.

[2] F. Chong, G. Carraro, and R. Wolter, "Multi-tenant data architecture," *Microsoft Corporation, MSDN Library*, Jun. 2006.

[3] S. Aulbach, T. Grust, D. Jacobs, A. Kemper, and J. Rittinger, "Multi-tenant databases for software as a service: Schema-mapping techniques," in *Proc. ACM SIGMOD Int. Conf. Management of Data*, Vancouver, Canada, 2008, pp. 1195–1206.

[4] M. Armbrust, A. Fox, R. Griffith, A. D. Joseph, R. Katz, A. Konwinski, G. Lee, D. Patterson, A. Rabkin, I. Stoica, and M. Zaharia, "A view of cloud computing," *Communications of the ACM*, vol. 53, no. 4, pp. 50–58, 2010.

[5] OWASP Foundation, "OWASP Top 10:2021 Web Application Security Risks," 2021. [Online]. Available: https://owasp.org/Top10/

[6] OWASP Foundation, "OWASP API Security Top 10:2023," 2023. [Online]. Available: https://owasp.org/www-project-api-security/

[7] Verizon, "2023 Data Breach Investigations Report (DBIR)," *Verizon Business*, 2023.

[8] IBM Security and Ponemon Institute, "Cost of a Data Breach Report 2023," *IBM*, 2023.

[9] Amazon Web Services, "SaaS tenant isolation strategies: Building SaaS solutions on AWS," *AWS Whitepaper*, Aug. 2020.

[10] J. H. Saltzer and M. D. Schroeder, "The protection of information in computer systems," *Proceedings of the IEEE*, vol. 63, no. 9, pp. 1278–1308, Sep. 1975.

[11] J. A. Goguen and J. Meseguer, "Security policies and security models," in *Proc. IEEE Symposium on Security and Privacy*, Oakland, CA, USA, 1982, pp. 11–20.

[12] M. Stonebraker and E. Wong, "Access control in a relational data base management system by query modification," in *Proc. ACM Annual Conference*, San Diego, CA, USA, 1974, pp. 409–417.

[13] A. Schiller, B. Schlosser, T. Bodenstein, and B. Schatz, "Native support of multi-tenancy in RDBMS for SaaS," in *Proc. 14th Int. Conf. Extending Database Technology (EDBT)*, Uppsala, Sweden, 2011, pp. 515–520.

[14] V. Narasayya and S. Chaudhuri, "Multi-tenant data services in the cloud," in *Proc. ACM SIGMOD Int. Conf. Management of Data*, New York, NY, USA, 2013, pp. 1231–1236.

[15] C.-P. Bezemer and A. Zaidman, "Multi-tenant SaaS applications: Maintenance dream or nightmare?" in *Proc. Joint ERCIM Workshop on Software Evolution (EVOL) and Int. Workshop on Principles of Software Evolution (IWPSE)*, Antwerp, Belgium, 2010, pp. 88–92.

[16] J. Kabbedijk, C.-P. Bezemer, S. Jansen, and A. Zaidman, "Defining multi-tenancy: A systematic mapping study on the academic and the industrial perspective," *J. Systems and Software*, vol. 100, pp. 139–148, 2015.

[17] D. Jacobs and S. Aulbach, "Ruminations on multi-tenant databases," in *Proc. Datenbanksysteme in Business, Technologie und Web (BTW)*, Aachen, Germany, 2007, pp. 514–521.

[18] C. D. Weissman and S. Bobrowski, "The design of the force.com multitenant internet application development platform," in *Proc. ACM SIGMOD Int. Conf. Management of Data*, Providence, RI, USA, 2009, pp. 889–896.

[19] C. J. Guo, W. Sun, Y. Huang, Z. H. Wang, and B. Gao, "A framework for native multi-tenancy application development and management," in *Proc. 9th IEEE Int. Conf. E-Commerce Technology and 4th IEEE Int. Conf. Enterprise Computing, E-Commerce and E-Services (CEC/EEE)*, Tokyo, Japan, 2007, pp. 551–558.

[20] R. Krebs, C. Momm, and S. Kounev, "Architectural concerns in multi-tenant SaaS applications," in *Proc. 2nd Int. Conf. Cloud Computing and Services Science (CLOSER)*, Porto, Portugal, 2012, pp. 426–431.

[21] PostgreSQL Global Development Group, "Row security policies," *PostgreSQL 17 Documentation*, 2024. [Online]. Available: https://www.postgresql.org/docs/17/ddl-rowsecurity.html

[22] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, "Role-based access control models," *IEEE Computer*, vol. 29, no. 2, pp. 38–47, Feb. 1996.

[23] D. F. Ferraiolo, R. Sandhu, S. Gavrila, D. R. Kuhn, and R. Chandramouli, "Proposed NIST standard for role-based access control," *ACM Trans. Information and System Security*, vol. 4, no. 3, pp. 224–274, Aug. 2001.

[24] C. Richardson, *Microservices Patterns: With examples in Java*. Shelter Island, NY, USA: Manning, 2018.

[25] S. Newman, *Monolith to Microservices: Evolutionary Patterns to Transform Your Monolith*. Sebastopol, CA, USA: O'Reilly Media, 2019.

[26] N. Dragoni, S. Giallorenzo, A. Lluch Lafuente, M. Mazzara, F. Montesi, R. Mustafin, and L. Safina, "Microservices: Yesterday, today, and tomorrow," in *Present and Ulterior Software Engineering*, M. Mazzara and B. Meyer, Eds. Cham, Switzerland: Springer, 2017, pp. 195–216.

[27] A. Georges, D. Buytaert, and L. Eeckhout, "Statistically rigorous Java performance evaluation," in *Proc. 22nd ACM SIGPLAN Conf. Object-Oriented Programming Systems, Languages, and Applications (OOPSLA)*, Montreal, Canada, 2007, pp. 57–76.

[28] T. Kalibera and R. Jones, "Rigorous benchmarking in reasonable time," in *Proc. Int. Symp. Memory Management (ISMM)*, Seattle, WA, USA, 2013, pp. 63–74.

[29] T. Mytkowicz, A. Diwan, M. Hauswirth, and P. F. Sweeney, "Producing wrong data without doing anything obviously wrong!" in *Proc. 14th Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, Washington, D.C., USA, 2009, pp. 265–276.

[30] D. J. Schuirmann, "A comparison of the two one-sided tests procedure and the power approach for assessing the equivalence of average bioavailability," *J. Pharmacokinetics and Biopharmaceutics*, vol. 15, no. 6, pp. 657–680, Dec. 1987.

[31] D. Lakens, "Equivalence testing for psychological research: A tutorial," *Advances in Methods and Practices in Psychological Science*, vol. 1, no. 2, pp. 259–269, Jun. 2018.

[32] Red Hat, Inc., "Hibernate ORM 6.6 User Guide: Multi-tenancy," 2024. [Online]. Available: https://docs.jboss.org/hibernate/orm/6.6/userguide/html_single/Hibernate_User_Guide.html

[33] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," *IETF RFC 7519*, May 2015.

[34] T. Lodderstedt, J. Bradley, A. Labunets, and D. Fett, "Best current practice for OAuth 2.0 security," *IETF RFC 9700*, Jan. 2025.